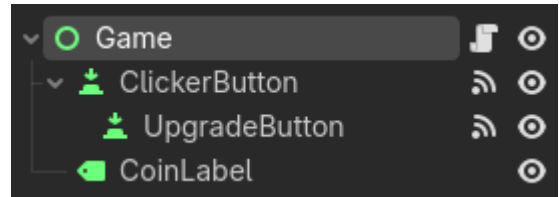


Restructuring for P2

Last lesson, we built a working game, but if you recall what we talked about today, you'll find that our game wasn't structured properly.

Current structure

Let's look at our structure now



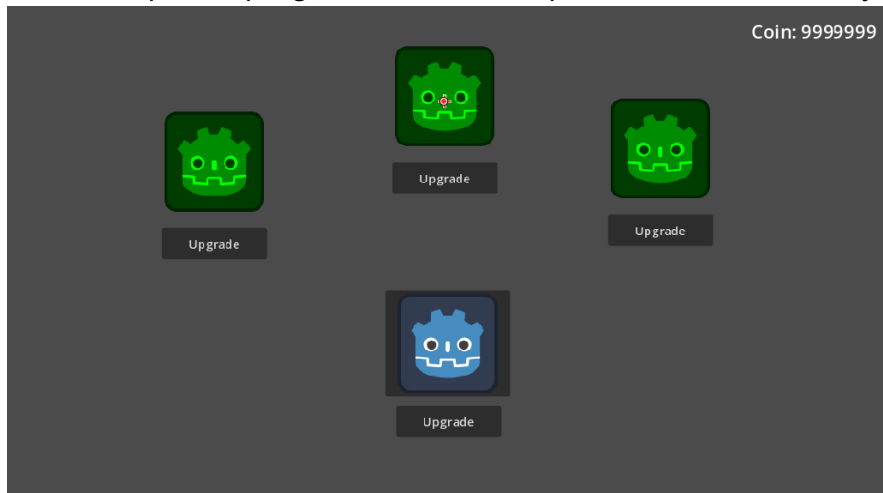
See the problem?

Everything is now all handled within the GameManager script. This is called a **monolithic design**, where everything is bundled together inside one object/script. While it might work for small projects like this, it's not a habit you want to have as a game developer.

As projects grow, they require dozens of interconnecting objects and scripts. And monolithic design scales poorly... They make games difficult to expand and debugging a nightmare when something breaks.

The Fix Part I - Structure

Let's fix it by decoupling the functions. Ask yourself: what are the objects in the game? And who owns what?

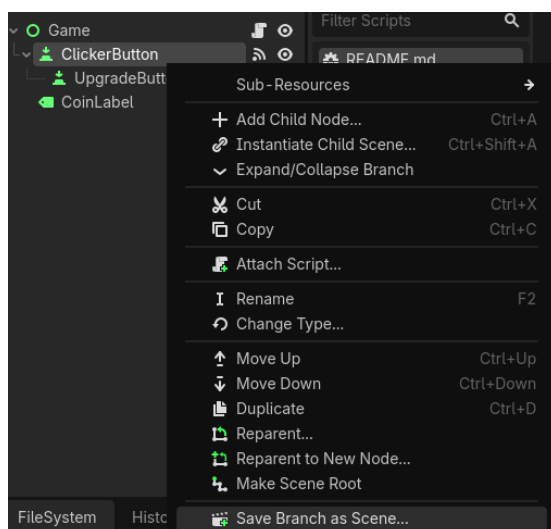


Game Manager
Coin count
UI display

Clicker
Clicker strength
Button click

Generator (for later)
Generator strength
Timer for generating

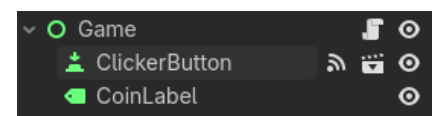
So yes, we should also make our clicker a separate game object (packed scene).



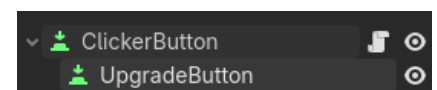
Fortunately, there is a quick shortcut in Godot.

Right-click on the clicker node > Save Branch as Scene... > Give it a name (e.g. "clicker.tscn")

You'll see it become a packed scene with the scene icon. This sets up the structure for us to decouple the script



Press the scene icon to open the Clicker object to edit, and let's attach a script to it.



The Fix Part II - Script

On the right is the all-in-one script we have now. What should we pull out to the clicker script? What should stay in game manager?

Think about how to make a clean structure and give it a try before continuing...

Here are the changes:

- Lines 3, 5: stay in GameManager
- Line 4: move to Clicker
- Lines 16-18: move to Clicker
- Lines 8-13 are more complicated because the actions (add coin and update UI) are all in GameManager, but the trigger for the actions should be sent by Clicker. Therefore, we need to wire the signals like this:

Button Pressed → Custom Clicker Signal → GameManager

```
1 extends Control
2
3 var coin: int
4 @export var clicker_strength: int
5 @onready var coin_label: Label = $CoinLabel
6
7 # When clicker is clicked
8 func _on_button_down() -> void:
9     # Add coins
10    coin += clicker_strength
11
12    # Update UI
13    coin_label.text = "Coin: " + str(coin)
14
15 # When upgrade button is pressed
16 func _on_upgrade_button_pressed() -> void:
17     # Double strength
18     clicker_strength = clicker_strength * 2
```

So we get the following structure:

Clicker (Button)

- ↳ pressed → root
- UpgradeButton
 - ↳ pressed → root

GameManager

- CoinLabel
- Clicker (Packed Scene)
 - ↳ (custom) clicked → root
- Generator (Packed Scene - later)
 - ↳ (custom) generated → root

Coding-wise, here's the pseudocode to get you started:

```
clicker.gd
var strength
signal clicked(int)

on_pressed():
    # Emit signal with current strength
    clicked.emit(strength)
    # Other juicy stuff here too

on_upgrade_pressed():
    # Double strength
```

```
game_manager.gd
var coin
Node $CoinLabel

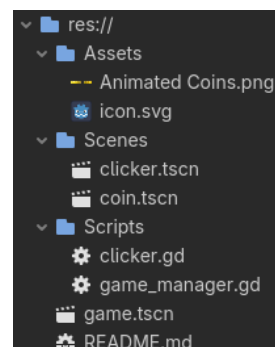
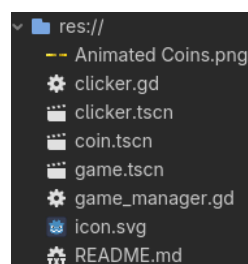
on_clicker_signal():
    # Add coin
    # Update UI
```

And voilà! We have a clean, modular structure that allows us to expand easily. You can easily make another clicker and just connect the signals, or even move on to create the passive generators. Now that we have a clean scene, let's also clean up our folder so everything looks neat.

Folder Clean Up

There are many ways to organize your project folders, and much of it comes down to personal preference. I'll share my structure, but feel free to adapt it.

Whichever structure you choose, the main goal is keeping files categorized and easy to find as your assets and scenes grow.



Game Juice Examples

Spawning Coins

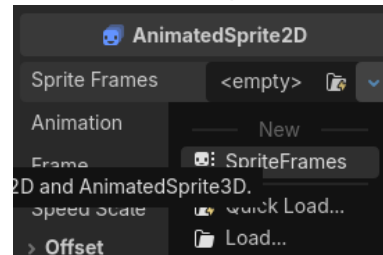
We finished importing the animation in class last week. Here's a quick recap:

1. Set up the nodes

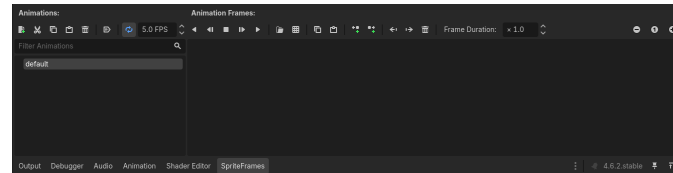
Coin (Rigidbody2D)

- CollisionShape2D [Shape:Circle]
- AnimatedSprite2D [SpriteFrame:Step 2]

2. Create a new Sprite Frame



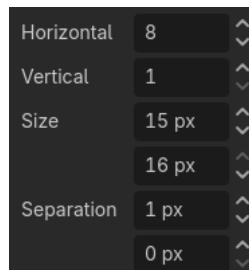
3. Use the bottom panel to configure the animation



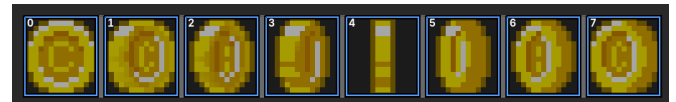
1) Use  to choose the spritesheet

2) Use the following settings if you're using my coin sprite.

Change this to match your spritesheet if you used another one.



3) Click and drag to select all frames



4) I'm also changing it to 15 FPS so it turns faster.

5) You can preview it in edit mode with 

6) Make sure these are turned on  The left one makes it autoplay on load, and the right one loops it over and over.

Now, save the scene (e.g. "coin.tscn") and we can spawn it in the Clicker script.

 clicker.gd

```
...
# Here's the line to read a packed scene, you can drag the scene file into the
# script to automatically generate the path like this ↓
@onready var coin_scene: PackedScene = load("res://Scenes/coin.tscn")
...
on_pressed():
    # Emit signal with current strength
    # Spawn the packed scene
    var c = coin_scene.instantiate() # Create the coin, and store a temp reference c
    add_child(c)                    # Add it to the tree so it shows up
    c.global_position = get_global_mouse_position() # Set the position to the cursor
...
```

Test it out, and you'll see the coins spawn! However, they currently pile up forever and never get destroyed, even out of sight. Leaving unused nodes running will slow down performance. So let's fix that too.

1. Add a VisibleOnScreenNotifier to the scene, create a script on the root, and connect the signal.

Coin (Rigidbody2D) 

- CollisionShape2D
- AnimatedSprite2D
- VisibleOnScreenNotifier2D
 - screen_exited → root

2. In the script  coin.gd

```
on_screen_exited():
    # Remove node
    queue_free()
```

And that does the trick.

Playing SFX

Sound effects play a big role in game juice, and it's rather easy to do. Just add an AudioPlayer to the Coin.

Coin (Rigidbody2D) - CollisionShape2D - AnimatedSprite2D - VisibleOnScreenNotifier2D → screen_exited → root - AudioStreamPlayer2D [Stream:SFX_Clip] [Autoplay:On]	And that's it!
--	----------------

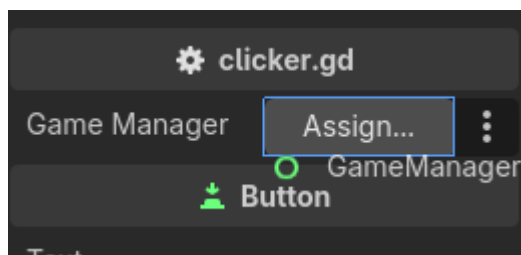
Fixing the Economy

As you might be wondering, there's a big glitch... the upgrades are free. Let's fix it by using an if-statement to check if the player has enough coins. For simplicity, I'll set the upgrade cost as 3 times the current strength.

Clicker (Button) → pressed → root - UpgradeButton → pressed → root	clicker.gd <pre>on_upgrade_pressed(): # If coin >= strength*3 # Double strength # Reduce coins by cost</pre>
--	--

Now we have a big problem. How do we access the player's coin? It's currently stored inside GameManager, which is a parent of our Clicker. We know we can easily access child nodes with \$, but we'll have to be more creative for getting other nodes.

To keep things simple without introducing advanced concepts yet (we'll cover the proper Autoload pattern in P3), we'll use an Inspector reference trick with what we already know. We can create an `@export var` that allows you to drag and assign a node in the inspector, and use that to get the reference to GameManager.



Add `@export var game_manager: Node` to `clicker.gd` and drag-drop assign the GameManager node in your main scene.

Then you can access the coin with `game_manager.coin`.

To reduce the cost from coin, you can reuse the custom signal with a negative value for cost.

Pause Here

I'll attach my [repo here](#) for you to see the complete code, but make sure you try it yourself first, and ask Google or AI, or message us if you're stuck. Once you're finished restructuring and fixing the mechanic, you should have the complete clicker function working, and can move on to adding the generator.

Generator Recipe

Let's apply what you learned by creating the generator using the recipe below:

Generator (Control) - TextureRect [Texture:GeneratorSprite] (Show the image, no interaction needed) - UpgradeButton → pressed → root - Timer [Autostart:On] → timeout → root	generator.gd <pre>var strength signal coin_generated(int) → GameManager on_upgrade_pressed(): # Same as Clicker on_timer_timeout(): # Emit signal # Effects (coin, sound)</pre>
---	--

That's it for the whole functional part of the game, and make sure you try it before our lesson on Thursday. We will have a few minutes to clarify any questions before moving on to the history of games, and you can work on adding aesthetics and juice to this over the next few weeks.